Project 2: The Librarian

CS496 AI Engineering, Fall 2026. A retrieval-augmented question answering service over Tunisian laws, in Arabic and French, with citations. Draft of 3 Nov 2026. Sections marked **[TO FILL]** depend on runs that are not done yet; nothing in them is estimated.

1. The corpus and its users

Corpus. Laws published in the *Journal Officiel de la République Tunisienne* (Imprimerie Officielle, iort.gov.tn), 2013 to 2026: **171 Arabic texts and 179 French texts** (134 laws exist in both languages), about 12 million characters, scraped with `src/ingest/iort_scrape.py`. It is real (the legal texts in force), public (no permission needed, no personal data), larger than any prompt, and every fact can be checked against an article. Loan and treaty ratifications were left out on purpose (about 450 of the 635 laws the site lists for 2013 to 2026); budget and finance laws are the biggest files and are mostly tables. Texts before 2013 could not be used: their PDFs use legacy fonts and extract as junk.

Users. Law students, lawyers, journalists and citizens who want the text of a rule, a delay, an authority or a penalty, in either language. The questions the system must handle look like *"in how many days must the Instance announce final election results?"* or *"can a candidate excluded by the parliamentary commission appeal?"*. Classmates will try the deployed service and vote on answers (section 5).

Evaluation data. 124 questions (`data/eval/qa.jsonl`): 50 facts, each asked in Arabic and in French, plus 12 questions the corpus cannot answer, in both languages. Each answerable question has a reference answer, the gold law and article, and the evidence passage. A second set of 50 **paraphrased** questions (25 facts x 2 languages, `qa_hard.jsonl`) avoids the laws' own wording and law numbers: the first set flatters keyword search, the second is the honest test. All were written by reading the French and Arabic articles side by side.

2. Pipeline design

Stage	Choice	Why
Chunking	One chunk per legal article (الفصل / Article), tiny articles merged (<200 chars), long ones split at 1,200 chars with 150 overlap; each chunk starts with "law reference, article"	A citation must name a law and an article. 15,831 chunks, median 1,001 chars
Index	Flat files: `embeddings.npy` (15,831 x 1,536, float16, 47 MB) + token lists, brute-force cosine	At 16k chunks a matrix product is milliseconds; a vector database would be one more service and no accuracy gain
Keyword search	BM25 with Arabic normalisation (alef variants, diacritics, tatweel, definite article) and French accent stripping	Legal questions often quote numbers and names
Vector search	OpenAI `text-embedding-3-small` (index built in 166 s for $0.135); local `multilingual-e5-small` as comparison (57 min on CPU)	Multilingual, so a French question finds the Arabic article and vice versa
Fusion	Reciprocal rank fusion (k=60) of the two top-30 lists	No score calibration needed
Reranking	Cross-encoder `mmarco-mMiniLMv2` on the top 30	Largest single accuracy gain, and the largest cost in time
Generation	`gpt-4o-mini`, temperature 0, answers only from the 5 numbered chunks, a citation `[n]` after every claim, reply in the question's language, "I don't know" otherwise	Faithfulness over fluency
Abstention	The model's instruction plus an optional reranker-score threshold (**[TO FILL]**: tuned value)	

3. Retrieval results and the ablation

A hit means the retrieved chunk is from the gold law and contains the gold article (matched by article number or evidence text, so Arabic and French chunks both count). Index: article chunking, OpenAI embeddings. With n = 100 (main) and n = 50 (paraphrased), differences under about 8 points are within noise.

search	main recall@1	main recall@5	main MRR	paraphrased recall@1	paraphrased recall@5	paraphrased MRR
keyword only (BM25)	87%	93%	0.898	12%	32%	0.207
vector only	73%	92%	0.817	34%	66%	0.481
hybrid (RRF)	84%	94%	0.894	26%	58%	0.386
hybrid + reranker	**97%**	**100%**	**0.983**	**52%**	**76%**	**0.612**

What each part added. On the main set the reranker lifts recall@1 from 84% to 97% and recall@5 to 100%; hybrid search adds little over BM25 alone because the questions reuse the articles' words. On the paraphrased set keyword search collapses (32%), the vector search carries the system (66%), and **hybrid is worse than vector alone** (58%): BM25 pushes wrong articles into the fused list. The reranker recovers most of it (76%), but recall@10 is also only 78%, so the remaining misses never reach the reranker: the candidate search is the limit there.

Chunking (e5 embeddings, recall@1 / MRR): article chunks beat 1,200-character fixed windows at the top ranks on both sets, for BM25 (87% vs 78%), vector (73% vs 59%) and the reranked system (97% vs 94% main; 52% vs 46% paraphrased). Recall@5 is similar (the right text is nearby either way), so article chunking mostly helps put the right article first, and it makes citations exact.

Embeddings. OpenAI and local e5 are close overall (reranked recall@5 on paraphrased questions: 76% vs 74%). They differ by language: vector-only recall@5 on paraphrased questions is 88% (French) and 44% (Arabic) with OpenAI against 60% and 68% with e5. With 25 facts per language this is a hint, not a result. The OpenAI index is what is deployed: it builds in 3 minutes instead of an hour and the service needs no local embedding model.

Latency. BM25 66 ms, vector 150 ms (e5, local) to 330 ms (OpenAI, one API call), hybrid 160-490 ms, and **hybrid + reranker 7-9 s per query** on this 8-core CPU, which dominates the whole request (section 5).

4. Generation results [TO FILL: generation eval, not run yet]

`eval/generation_eval.py` computes, for the 124 questions: answer correctness (LLM judge against the reference answer; judge `gpt-4.1-mini`), faithfulness (every claim supported by a cited chunk), citation accuracy (cited chunk is the gold article), abstention on the 24 unanswerable questions, and over-abstention. It is written and unit-tested but its first real run is still to do. A table of these numbers, by language, goes here, with a few judged failures.

Smoke test of the deployed pipeline (5 questions, not a measurement): both off-topic questions (minimum wage, capital of France) were refused; legal questions were answered with the right article cited, at a cost of about $0.0002 and 7.5-8.8 s per request.

5. Week 8: what changed and what it saved [TO FILL]

Baseline to beat (same test set before and after): reranked search 7-9 s of an 8-9 s request; generation about $0.0002 per request. Prepared options, all off by default and measured one at a time: fewer rerank candidates (30 to 10), shorter rerank input, skipping the reranker when the first search is confident, a smaller generation model (`gpt-5-nano`), fewer chunks in the prompt, caching repeated questions, embeddings with fewer dimensions. The rule is a 50% cut in cost or p95 latency with at most 2 points of accuracy lost.

6. Known failure cases and cost model

Failure cases (all seen in this project):

- **Amendments are not merged.** The corpus holds each law as published. Asking for the delay in which election results are announced returned the 2024 amending law (Court of Cassation, presidential elections), not the 2014 law the question was written from. The answer is faithful to its source, but the user may not realise the article was amended.
- **Tables.** Budget and finance laws extract as loose lines of numbers; questions about a figure in a table are not answerable reliably, and these files dominate the chunk count.

- **Parallel texts disagree.** Article numbers in an *amending* law can differ between the Arabic and French versions, and some Arabic files contain the end of a neighbouring law (the site's PDFs span gazette pages).
- **Near-identical laws.** Several mutuelle laws differ only in a minister's name; they are found by the law number in the question, not by meaning.
- **Paraphrased questions** lose most of keyword search, and Arabic vector search looked weaker than French with the OpenAI embeddings.
- **Coverage.** 2013-2026 only, none for 2022, and the Arabic side is thinner than the French.
- **Latency.** The reranker makes every request 7-9 s on CPU.

Cost model (preliminary: measured on 7 real service requests; `eval/cost_model.py` recomputes it from the generation baseline, full table in `results/cost_model.md`; list prices to be re-checked):

Item	Value
Index build, once	$0.135 (6.76 M tokens), 166 s
Question embedding	about 25 tokens: $0.0000005 per request
Generation (`gpt-4o-mini`, 5 chunks, about 1,000 prompt tokens)	$0.00023 per request (refused Arabic questions had longer prompts)
Per request	**about $0.00023, 7.6 s p50 / 8.8 s p95**
Today: 100 requests per day (a class demo)	$0.68 per month in model fees; free CPU Space, 1 worker
At 100 times: 10,000 requests per day	$68 per month in model fees; 4 workers at 25% utilisation, about $120 per month if a worker costs $30 (an assumed price)

Hosting grows with the reranker's CPU time, not with model fees, which is why it is the Week 8 target.

[TO FILL] after the generation eval and the Week 8 changes: measured cost per request with the judge's numbers, p50/p95 latency before and after, and feedback from real users.